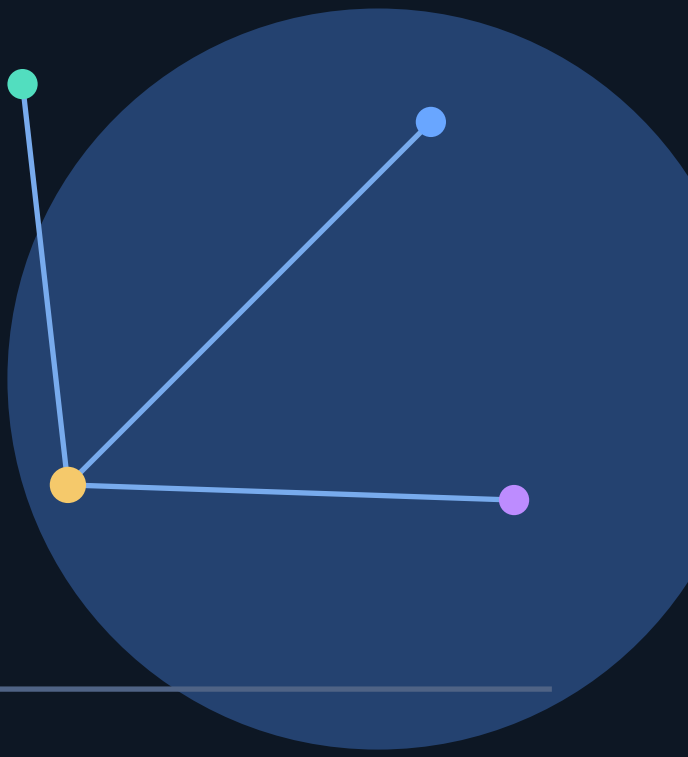
LINEAR ALGEBRA | PROJECT 12

# BasisBridge

## Project explanation & viva guide

Interactive Multiple Coordinate System Transformation Lab



**A practical guide to presenting the project,**
**explaining the mathematics, and answering viva questions.**

Includes a worked example, application walkthrough, implementation notes, verification strategy, presentation script, and 25 questions with answers.

Prepared for an undergraduate Linear Algebra and Vector Calculus mini-project.

Use the explanations in your own words and be ready to demonstrate the app live.

# 1. Project at a glance

A concise, faculty-friendly explanation of what BasisBridge does and why it matters.

## Project aim

To transform an initial vector successively through three valid coordinate bases, show every intermediate coordinate vector and change-of-basis matrix, then reverse the transformations and verify that the original vector is recovered.

## Problem being solved

A geometric vector can be described using many different bases. Students often confuse the vector itself with the coordinate column used to describe it. BasisBridge makes the distinction visible: the geometric arrow stays fixed while its numerical coordinates change with the chosen basis.

## What the application does

- Accepts a 2D or 3D vector, plus three editable basis matrices.
- Checks dimensions, finite input values, and basis invertibility before doing the conversion.
- Computes the standard-to-basis-1 conversion and the B1-to-B2 and B2-to-B3 coordinate changes.
- Displays matrices, multiplication steps, intermediate coordinates, and a reverse round-trip check.
- Provides 2D animated frames, a rotatable and zoomable 3D view, local experiment history, and PDF/CSV exports.

### One-sentence summary

BasisBridge is an offline-ready educational lab that changes coordinate descriptions, not the underlying geometric vector, and checks its work by reversing the changes.

## Learning outcomes

The project demonstrates basis and dimension, linear independence, change of basis, matrix multiplication, matrix representation of linear maps, inverse matrices, numerical tolerance, and round-trip verification.

# 2. Mathematical foundations

Know what each symbol means before describing the software.

## Vectors, bases, and dimension

A vector space is a collection of vectors that can be added and scaled. A basis is a linearly independent set that spans the whole space. The number of vectors in a basis is the dimension. Therefore a basis of R^2 has two independent vectors and a basis of R^3 has three.

BasisBridge stores those basis vectors as the columns of a square matrix B. For example, if b1 = [1, 1] and b2 = [1, -1], then the corresponding basis matrix is:

```
B2 = [ 1   1 ]\n     [ 1  -1 ]
```

## A vector versus its coordinates

The geometric vector v is independent of the coordinate system. Its coordinate column [v]_B contains the coefficients needed to combine the basis vectors and make v. In column notation, v = B [v]_B. If B is invertible, solving this equation gives [v]_B = B^-1 v.

```
v = B [v]_B      therefore      [v]_B = B^-1 v
```

## Changing coordinates between two bases

Suppose [v]_source gives a vector in basis B_source. First reconstruct the standard vector using B_source, then express it in B_destination. Combining the two operations gives the change-of-basis matrix:

```
P_destination<-source = B_destination^-1 B_source\n[v]_destination =
P_destination<-source [v]_source
```

**Important direction convention**

The rightmost matrix acts first. The destination basis is inverted; the source basis is not. Keep vectors as columns and multiply matrices from the left.

## Why invertibility matters

A square matrix is invertible exactly when its columns are linearly independent. If the columns are dependent, they do not form a basis, coordinates are not unique, and the basis inverse does not exist. The app rejects singular or numerically unstable bases rather than showing a misleading result.

# 3. Worked default example

These are the values loaded when the lab starts. Practice explaining every line.

## Inputs and coordinate bases

```
v = [5, 4]\nB1 = [ 1  0 ]    B2 = [ 1   1 ]    B3 = [ 2  0 ]\n     [ 0  1 ]
[ 1  -1 ]         [ 0  1 ]
```

B1 is the standard basis. The determinant of B2 is -2, and the determinant of B3 is 2. All three determinants are nonzero, so all three bases are valid.

## Step 1: Standard coordinates into Basis 1

```
B1^-1 = I\n[v]_B1 = B1^-1 v = [5, 4]
```

## Step 2: Basis 1 into Basis 2

```
B2^-1 = [ 0.5   0.5 ]\n         [ 0.5  -0.5 ]\nP21 = B2^-1 B1 = B2^-1\n[v]_B2 = P21
[v]_B1\n       = [ 0.5   0.5 ] [ 5 ] = [ 4.5 ]\n         [ 0.5  -0.5 ] [ 4 ]   [ 0.5
]
```

Check directly: [4.5, 0.5] means 4.5 times [1, 1] plus 0.5 times [1, -1], which equals [5, 4].

## Step 3: Basis 2 into Basis 3

```
B3^-1 = [ 0.5  0 ]\n         [ 0    1 ]\nP32 = B3^-1 B2 = [ 0.5   0.5 ]\n
        [ 1    -1 ]\n[v]_B3 = P32 [v]_B2 = [ 2.5, 4 ]
```

## Reverse and verify

```
P32^-1 [v]_B3 = [4.5, 0.5] = [v]_B2\nP21^-1 [v]_B2 = [5, 4] = [v]_B1\nB1 [v]_B1 =
[5, 4] = original v\nMaximum absolute error = 0; verification = PASS
```

### Quick geometric check

The Basis 3 coordinates [2.5, 4] mean 2.5 times [2, 0] plus 4 times [0, 1] = [5, 4]. The original vector did not move.

# 4. How the application is built

Explain the implementation as a sequence of understandable parts.

## Technology choices

- React and TypeScript build the interactive user interface and give the matrix data explicit types.
- Vite serves the development app and produces the optimized production build.
- A custom numerical engine performs pivoted Gauss-Jordan inversion, determinants, matrix products, and vector products. No API or cloud calculation is used.
- Three.js renders the interactive 3D coordinate scene; the 2D scene uses SVG.
- Lucide React supplies interface icons; jsPDF generates reports; browser localStorage stores experiments on the device.

## Calculation pipeline

```
1. Parse and validate vector and matrix entries.\n2. Check three square bases of the selected dimension.\n3. Find each determinant and inverse; reject an unstable pivot.\n4. Compute [v]_B1 = B1^-1 v.\n5. Compute P21 = B2^-1 B1 and [v]_B2 = P21 [v]_B1.\n6. Compute P32 = B3^-1 B2 and [v]_B3 = P32 [v]_B2.\n7. Reverse with P32^-1, then P21^-1, then reconstruct with B1.\n8. Compare reconstructed and original coordinates against tolerance 1e-9.
```

## Numerical stability and tolerance

Computers use floating-point numbers, so operations may introduce tiny rounding errors. The inversion routine chooses the largest available pivot in a column and rejects pivots that are too small relative to the matrix scale. The app computes the maximum absolute difference, max_i |v_reconstructed,i - v_original,i|, and passes if it is at most 1e-9.

## User-facing validation

Blank, non-finite, wrong-dimension, singular, and numerically unstable inputs produce an actionable error. A rejected input disables run/save actions so an invalid basis cannot be treated as a successful transformation.

# 5. App demonstration and presentation

A short, repeatable walkthrough for a faculty demonstration.

## Suggested 2-minute presentation

“Our project is BasisBridge, an interactive multiple coordinate system converter. Its goal is to take a vector through three bases and verify the result by reversing the transformations. I will use v = [5, 4] with an identity basis, an oblique basis, and a scaled basis.”

- Start at Dashboard and introduce the goal and mathematical concepts.
- Open Input Configuration. Point out that basis vectors are matrix columns, then identify the default example.
- Run the transformation. Explain that [v]_B1 = B1^-1 v and that P21 = B2^-1 B1.
- Read out [v]_B2 = [4.5, 0.5] and [v]_B3 = [2.5, 4]. Show the matrix explorer for the actual matrix products.
- Open visualization. Emphasize that the gold arrow is the unchanged geometric vector; the other colors identify bases.
- Open Reverse Verification. Show reconstruction [5, 4], maximum error, and PASS.
- Briefly mention that singular bases are rejected, experiments stay in local browser storage, and CSV/PDF files can be exported.

## If asked to demonstrate 3D

Select 3D in Input Configuration. The included 3D preset supplies three invertible matrices and a three-component vector. Run it, open the 3D visualization, drag to rotate, and scroll to zoom. The coordinates and verification are recomputed by the same dimension-general matrix engine.

## Presentation advice

- Say “coordinates change, vector stays the same” more than once; it is the key idea.
- Explain one matrix product clearly instead of reading every screen aloud.
- When asked about accuracy, give the tolerance (1e-9) and the maximum-error comparison.
- If you alter a basis, first predict whether its determinant is zero, then observe whether the validator accepts it.

# 6. Viva questions and answers (1-9)

Short answers are a starting point. Add the relevant example when answering.

**Q. What is the aim of BasisBridge?**

A. To transform a vector successively through three valid bases, display intermediate coordinates and matrices, and recover the original vector through reverse transformations.

**Q. What is a basis?**

A. A linearly independent set of vectors that spans the vector space. Every vector in the space has a unique coordinate representation in that basis.

**Q. What is the dimension of a vector space?**

A. The number of vectors in any basis of the space. R^2 has dimension 2; R^3 has dimension 3.

**Q. How are basis vectors stored in this project?**

A. As the columns of a square basis matrix. This matches the equation v = Bc for the coordinate column c.

**Q. What is the difference between a vector and its coordinate column?**

A. The vector is the geometric object; its coordinate column is a list of coefficients relative to a chosen basis. The same vector can have different coordinate columns.

**Q. How do you convert standard coordinates into basis B?**

A. Solve v = Bc. Since B is invertible, c = B^-1v, which the app writes as [v]_B = B^-1v.

**Q. What is the change-of-basis matrix from B1 to B2?**

A. P21 = B2^-1 B1. It converts coordinates in B1 into coordinates in B2: [v]_B2 = P21 [v]_B1.

**Q. Why is the destination basis inverted?**

A. We first turn the source coordinates into the geometric vector using B_source, then express that vector using the destination basis by multiplying by B_destination^-1.

**Q. Do coordinates change the geometric vector?**

A. No. Under a pure change of basis the geometric vector is unchanged; only the coefficients used to describe it change.

# 7. Viva questions and answers (10-18)

Be prepared to explain validation and reverse verification.

**Q. What does linearly independent mean?**

A. No vector in the set can be made from a linear combination of the others. Equivalently, the only combination that gives zero uses all-zero coefficients.

**Q. How is invertibility related to a basis?**

A. A square matrix is invertible exactly when its columns are linearly independent. Its columns then form a basis.

**Q. What happens if a basis matrix is singular?**

A. It has determinant zero and no inverse. The columns are dependent, so they do not form a basis and the coordinates may not be unique. The app stops the transformation.

**Q. What does the determinant tell you here?**

A. For a square basis matrix, a nonzero determinant indicates invertibility. Its magnitude also describes volume scaling; its sign indicates orientation.

**Q. What is a linear transformation?**

A. A map T that preserves addition and scalar multiplication: T(x + y) = T(x) + T(y) and T(ax) = aT(x). Matrix multiplication is a standard representation of a linear map.

**Q. Why do we use matrix multiplication?**

A. It composes linear operations and transforms coordinate columns. In P21 = B2^-1 B1, the source basis is applied first, followed by the destination inverse.

**Q. How does reverse transformation verify the forward calculation?**

A. It applies inverse coordinate changes in reverse order, reconstructs the vector in standard coordinates, and compares it with the original input.

**Q. Why is verification based on a tolerance instead of exact equality?**

A. Floating-point arithmetic rounds results. A small difference such as 1e-15 can be normal; a tolerance like 1e-9 allows harmless rounding while detecting meaningful errors.

**Q. What is numerical stability?**

A. How sensitive a calculation is to small input or rounding errors. A nearly singular matrix can amplify errors, so the app rejects pivots that are too small relative to matrix scale.

# 8. Viva questions and answers (19-25)

Questions about implementation, testing, visualization, and scope.

### Q. What inversion method is used?

A. A custom Gauss-Jordan elimination routine with partial pivoting: it chooses the largest-magnitude pivot in each column, normalizes the pivot row, and eliminates the other entries.

### Q. How does the app calculate a determinant?

A. It uses elimination with pivoting, multiplies the pivots, and accounts for row-swap sign changes.

### Q. How are dimensions validated?

A. The vector length must be 2 or 3; there must be exactly three basis matrices; and each must be a square matrix matching the chosen dimension.

### Q. How is the 3D view made interactive?

A. Three.js renders basis and vector arrows, grid, and axes. OrbitControls lets the viewer rotate and zoom the scene.

### Q. Where are experiment history entries stored?

A. In the browser localStorage on the current device. No account or remote service is needed; clearing browser storage can remove saved experiments.

### Q. What is tested automatically?

A. The test suite checks the required 2D example, a 3D example, reverse reconstruction, singular-basis rejection, invalid dimensions/non-finite values, and matrix multiplication dimensions.

### Q. What are limitations and possible future improvements?

A. The app uses JavaScript floating-point precision and a fixed practical tolerance. Future work could add condition-number estimates, customizable tolerances, import/export of saved experiment sets, and more basis presets.

### Q. Why is this more than a calculator?

A. It connects input, validation, intermediate matrices, coordinate explanations, visualization, learning content, history, exports, and round-trip verification in one reproducible experiment.

# 9. Final checklist and key takeaways

Use this page to rehearse just before the presentation.

## Five facts worth memorizing

- Basis vectors are columns: v = B[v]_B.
- Standard-to-basis coordinates: [v]_B = B^-1v.
- Source-to-destination change: P_destination<-source = B_destination^-1 B_source.
- A valid square basis has nonzero determinant and an inverse.
- The vector stays the same; the coordinates change; reverse reconstruction checks the result.

## Default example results

```
v = [5, 4]\n[v]_B1 = [5, 4]\n[v]_B2 = [4.5, 0.5]\n[v]_B3 = [2.5, 4]\nReconstructed v = [5, 4]\nMaximum error = 0; PASS
```

## Before you present

- Run npm run dev and open the local BasisBridge page.
- Keep the default example loaded and verify the Dashboard shows PASS.
- Know where Input Configuration, Matrix Explorer, Visualization, and Reverse Verification are in the sidebar.
- Be ready to make two columns identical to demonstrate a singular basis being rejected; restore the preset afterward.
- Do not describe a coordinate change as moving the geometric vector.

### Closing sentence

BasisBridge demonstrates that changing a basis changes a vector’s coordinate description, while matrix inverses make conversion reversible and numerical comparison verifies the computation.